

Bataille Naval en JAVA

-Grysan Alan
- Lamairi Mohamed Yassine
-Rezgui Léna
-Le Basnier Audrey

3 avril 2025

Table des matières:

1 Présentation de l'algorithme	1
1.1 Grille	1
1.2 Arborecence des fichiers du projet	2
1.3 Gestion des positions	3
1.4 Gestion des bateaux	3
1.5 Creation de la flotte	4
2 La gestion des joueurs	4
2.1 Amiral (joueurHumain)	5
2.2 IaFolle (modefacile)	5
2.3 Ia PseudoIntelligente (modenormale)	5
2.4 TERMINATOR(mode difficile)	6
3 Vue et Controleur	6
3.1 Vue et Controleur terminal	6
3.2 Vue et Controleur graphique.....	7
3.3 La fenêtre paramétrable	8

1 Présentation de l'algorithme

1.1 Grille

Le terrain de jeu est une grille de type *Grid* spécifié par sa largeur et sa hauteur. Chaque cellule de la grille est un objet de type *Case*, que ce soit une case de type *CaseMer* ou *pieceBateau*. Ces cases possèdent un symbole de type *String* qui permet de connaître leurs appartenance. Ainsi, lors de la création d'un bateau, ce dernier hérite du symbole de son propriétaire, de même pour chaque pièce qui compose le bateau. Les cases vide sont représentées par le symbole " ". Chaque joueur possède sa propre grille, et les coups sont enregistrés dans la grille via la méthode *coupTire*.

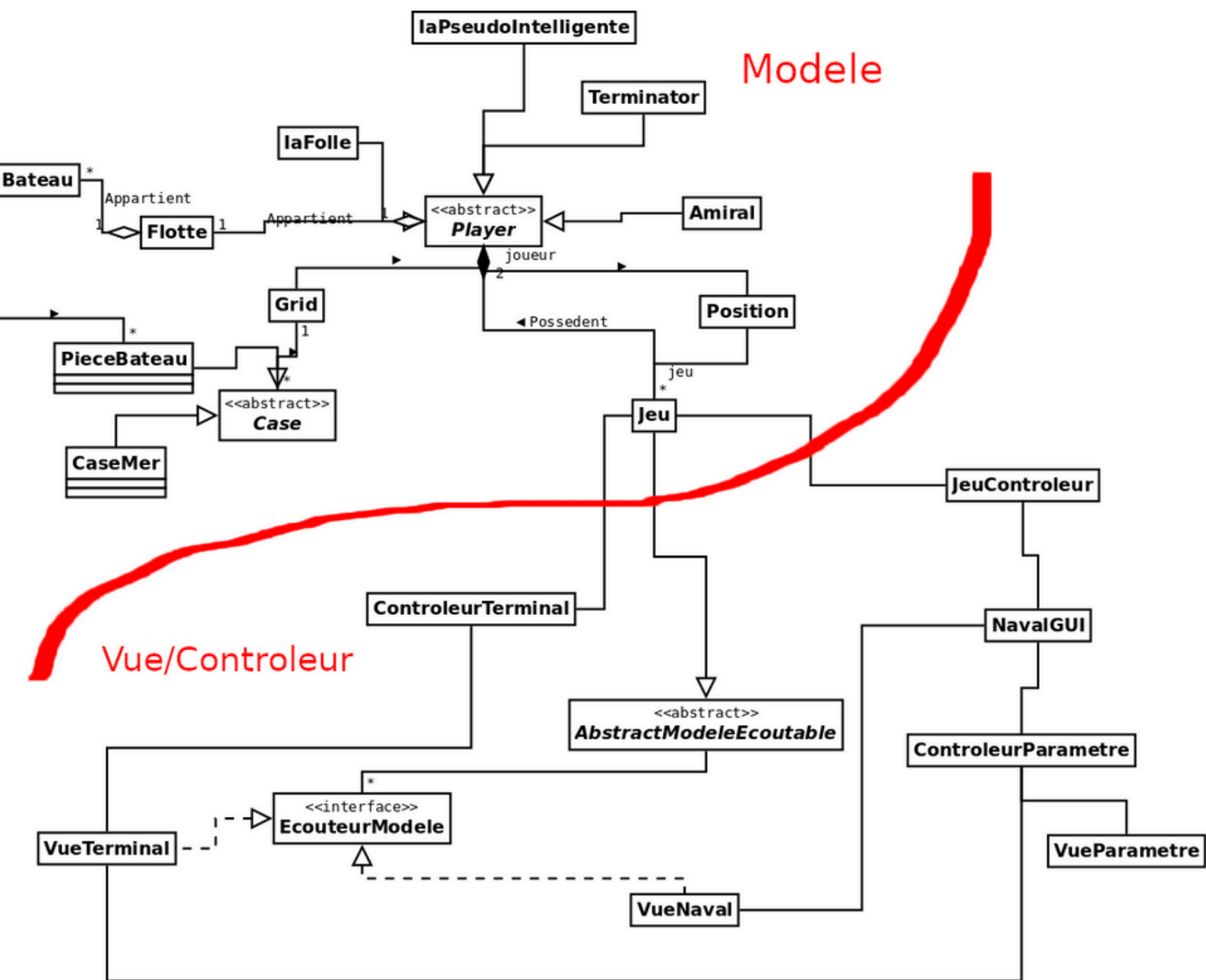


Figure 1 – Diagramme de classe du projet

1.3 Gestion des positions

Les positions sont gérées par la classe *Position*. Ces positions permettent de se repérer et de naviguer efficacement entre l'utilisateur et le code. Chaque position est constituée des coordonnées (x,y) et d'un symbole. Ce symbole est ce qui permet de facilement se repérer sur la grille.

$(0,0) \longleftrightarrow A1$ $(2,2) \longleftrightarrow C3$ $(9,9) \longleftrightarrow J9$

Figure 2 – Conversion des symboles

La position prend en compte le décalage de 1 entre les coordonnées et le numéro du symbole. Les conversions sont possible grâce aux méthodes *getPositionFromSymbol* et *getSymbolFromCoord* de la classe *Grid*.

1.4 Gestion des bateaux

Les bateaux sont des objets de type *Bateau*, qui sont spécifiés par leurs taille, leurs orientation (verticale ou horizontale) et leurs propriétaire. Seul la position de la tête du bateau importe, puisqu'avec cette position, la taille et l'orientation, on peut calculer la position du reste de la queue. Les bateaux existent sur la grille sous la forme d'une liste de *PieceBateau*. Cette liste constitue l'attribut *coque* du bateau. Également, ils possèdent l'attribut *estMort* qui est un booléen qui vérifie si le bateau est effectivement coulé.

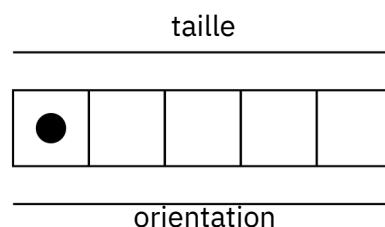


Figure 3 – Représentation d'un bateau

1.4 Creation de la flotte

Chaque joueur possède une flotte de bateaux, qui est une instance de la classe *Flotte*. La flotte est définie par sa taille (le nombre de bateau), et possède une liste *listeBateau* qui contient toutes les instances *Bateau* de la flotte. Lors de l'instanciation de la flotte, la méthode *remplisListeBateau* est appelée, et permet de créer les bateaux avec la méthode *creerBateau*. Cette dernière permet de placer la tête du bateau, dans une position valide. La position est considérée comme valide lorsque la potentielle queue qui suit ne dépasse pas de la grille et ne rencontre pas une autre pièce de bateau. Si la position de la tête est valide, le reste de la queue est créé. Une flotte est considérée détruite si tous les bateaux de sa liste ont coulé.

2 La gestion des joueurs

Une partie est tout le temps jouée par le joueur 1 (qui est un joueur humain), et le joueur 2 (qui est une ia parmi 3 choix). Les joueurs héritent d'une classe abstraite *Player*, et obtiennent les coups désirés via la méthode *action*. Un joueur est défini par son symbole (qui permet de le différencier de l'autre joueur) et sa flotte. Également, ils possèdent chacun les attributs *listeTirPossible* et *listePositionTir*, qui sont des listes de position de type *List<Position>*.

- l'attribut *listeTirPossible* contient les positions qui sont des coups valides. Cette liste est remplie lors de la création de l'instance du joueur, et contient toutes les positions qui ne sont pas associées à un bateau du joueur (pour éviter de se tirer dessus). Lorsque le joueur choisit un coup valide, la position associée au coup est retirée de la liste. Ainsi, le calcul des coups valides ne se fait qu'une fois, au début de la partie, et permet d'éviter de tirer 2 fois au même endroit puisque la position n'est plus dans la liste et donc invalide. — l'attribut *listePositionTir* contient les positions déjà tirées par le joueur.

Lorsqu'un coup est choisi, la position associée à ce coup est ajoutée à la liste. Cela permet de bien distinguer les coups entre les joueurs,

notamment lors de l'affichage graphique.

Bien que ces attributs soient communs entre les joueurs, chacun d'entre eux possède des subtilités sur la gestion des coups.

2.1 Amiral (joueurHumain)

Le joueur 1 est obligatoirement joué par un humain, qui est représenté par une instance de la classe *Amiral*. Le joueur se distingue par sa capacité à pouvoir choisir une position avec un clique sur la vue (lorsqu'on joue avec l'affichage graphique) ou alors directement en entrant un symbole associé à la position voulue (lorsqu'on joue avec le terminal). Si la position associée au clique est **null**, alors c'est que le joueur joue avec le terminal. Son symbole est "X".

2.2 Ia Folle (mode facile)

Un type du joueur 2, qui est une instance de *IaFolle*. Les coups choisis par ce joueur sont purement aléatoires, qu'elle pioche directement dans la liste de coups valides. Aucune réflexion, aucune stratégie, juste de la pure destruction. Son symbole est "O".

2.3 Ia Pseudo Intelligente (mode normale)

Ce joueur est une instance de la classe *IaPseudoIntelligente*. Elle a la particularité de "réfléchir" un minimum lors de la partie. Tant que son dernier tir n'a pas touché de bateau adverse, elle reste en mode recherche, ou elle tir de façon optimisée pour ne tirer aléatoirement que sur les positions impaires de la grille (bien sur, toujours selon la liste de coups possible). Cependant, lorsqu'elle touche une cible ennemi, elle passe en mode chasse. Dans ce mode, elle sauvegarde la position du dernier tir réussi, et remplit la liste *listePositionVoisin* qui sauvegarde les 4 positions alentour, en forme de croix (si ce sont des positions valides). Des lors, elle ne va tirer que sur ces positions sauvegardées, et ce jusqu'à que la liste soit vidée. Si en cours de route elle tire sur une autre pièce de bateau ennemi, les positions alentours sont également ajoutées à la liste *listePositionVoisin*, et ainsi de suite. Ainsi, lorsqu'elle touche un bateau, elle est garantit de le détruire dans les tours suivant.

Dans le pire des cas, le nombre de tour avant la destruction du bateau vaut : **$nbTour = 3 * t + 2$** et dans le meilleur des cas, il vaut : **$nbTour = t$** Avec t la taille du tableau. Cependant, avec cette Ia, le pire des cas est tout le temps le cas, puisqu'elle n'actualise pas sa liste *listePositionVoisin* dynamiquement, dans le sens où elle ne va pas suivre l'orientation du bateau, et continuer de tirer parmi les positions de la liste, et ce même lorsque le bateau est déjà détruit (d'où le "pseudo intelligente"). Son symbole est "@"

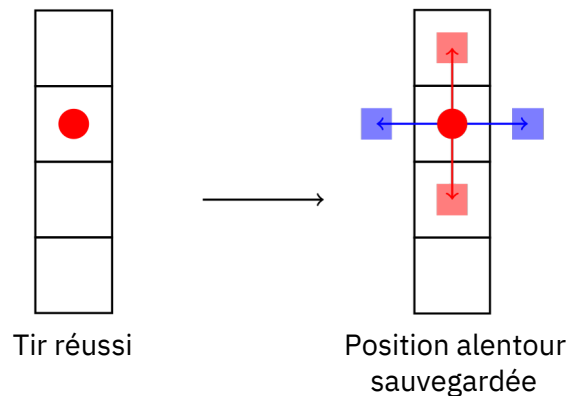


Figure 4 – Représentation de la stratégie de l'ia

2.4 TERMINATOR (mode difficile)

Le TERMINATOR n'est pas censé être battu. C'est une instance de la classe Terminator. Grâce à ses connaissances supérieures, il connaît précisément chaque position des bateaux adverses. Sa liste *listeTirPossible* contient directement chaque position des bateaux adverses. Si votre premier tir échoue, la partie est mathématiquement perdue. Son symbole est "#".

3 Vue et Controleur

Le joueur peut choisir si il veut jouer sur le terminal ou avec une interface graphique. Il y a pour chacun des 2 cas une vue et un controleur. La vue s'actualise à chaque tour d'un joueur, grâce à un écouteur *AbstractModeleEcoutable* ajouté au modèle. À chaque fois que la méthode *execute* est lancée par le modèle (qui correspond à un tour d'un joueur), le changement est notifié par la méthode *modeleMisAJour* de la vue, qui implémente l'interface *EcouteurModele*.

3.1 Vue et Controleur terminal

Pour une partie avec un terminal, des instances de *ControleurTerminal* et *VueTerminal* sont créées. Le joueur humain joue avec le clavier, en entrant un coup valide selon son symbole (A1, B5, ect...). La partie est fini lorsque la flotte d'un des 2 joueur est coulée.

3.2 Vue et Controleur graphique

Pour une partie avec une interface graphique, des instances de *JeuContrôleur* et *VueNaval* sont créées, ainsi que *NavalGUI*. Le joueur humain joue avec le clique de la souris. L'interface graphique se divise en 2 grilles, qui sont dessinées l'une après l'autre. Les tirs en rouge correspondent à un tir qui a touché, et ceux en bleu représentent les tirs ratés.

- Celle de gauche représente la grille du joueur. Elle n'est pas cliquable, et les bateaux du joueur ainsi que les tirs de l'ordi y sont représentés
- Celle de droite représente la grille de l'ordi. Elle est cliquable, et les tirs du joueur sont représentés la dessus. Lorsqu'un bateau adverse est coulé, sa silhouette se dessine

Lors du dessin de la grille, pour la flotte, il suffit de récupérer la liste de bateaux de la flotte du joueur. Avec la position connue de la tête de chaque bateau, sa taille et son orientation, on peut facilement calculer les coordonnées pour afficher le corps du bateau. Pour les tirs, on récupère la liste des coups déjà tirés par le joueur. Si, à cette position, la case de la grille contient une instance d'une pièce de bateau, alors un cercle rouge s'affiche, ou bleu sinon. Cela permet d'éviter de parcourir l'entièreté des 2 grilles à chaque dessin.

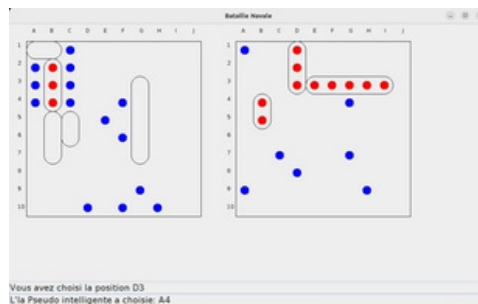


Figure 5 – Exemple de l'affichage

La gestion du clique se fait par le controleur. Lors de la création de la vue, on abonne à ce dernier le controleur. Lors du clique de la souris, la coordonnée est récupérée par la méthode *mouseClicked*. Après les calculs en fonction de la fenêtre, ces coordonnées sont vérifiées pour qu'elles soient à la fois dans les dimensions de la grille, et à la fois valides, en vérifiant si la coordonnée obtenue est bien dans la liste des coups possibles du joueur. Également, la vue est contenue dans le GUI, qui est une instance de la classe *NavalGUI*. Le GUI actualise dynamiquement les derniers coups joués par le joueur et l'ia à chaque tour, via le controleur.

3.3 La fenêtre paramétrable

Les parties sont personnalisable selon 3 paramètres, qui sont :

— **La taille de la grille** : Plutôt explicite, on peut choisir une petite grille (5x5 avec 2 bateau par joueur), une grille modeste (10x10 avec 5 bateaux par joueur) et une grande grille (15x15 avec 7 bateaux par joueur).

— **La difficulté** : L'Ia folle représente la difficulté facile, l'Ia Pseudo-Intelligente la difficulté normale, tandis que le TERMINATOR représente la difficulté difficile (ou plutôt impossible).

— **L'affichage** : qui peut soit être l'affichage graphique ou alors l'affichage par terminale.

Après les bonnes options choisies, la partie débute.

Conclusion

Ce projet de développement Java nous a offert une opportunité précieuse pour consolider notre maîtrise des principes fondamentaux de la programmation orientée objet et de l'architecture Modèle-Vue-Contrôleur (MVC).

Les défis rencontrés lors de l'implémentation nous ont permis d'approfondir notre compréhension :

- Gestion propre des dépendances entre composants
- Adaptation du pattern MVC